OSTRAVSKÁ UNIVERZITA
ÚSTAV PRO VÝZKUM A APLIKACE FUZZY MODELOVÁNÍ

7LLMA · TAHÁK PRO STUDENTY

Prompty: anatomie a techniky

Anatomie · syntaxe · techniky · malé modely · testování

k decku 7LLMA-2-prompty-v2.1 · stav 22. 9. 2026

Co je prompt

- **Jen text:** instrukce + kontext + vstup → výstup. Model nevidí nic než tokeny v okně – ne obrazovku, ne minulý chat, ne váš záměr.
- **Systémový prompt** = kdo model je a jak se chová; **uživatelský** = co chci teď. U malých modelů je rozdíl slabší → instrukce raději do obou.

Špatný prompt nepoznáte podle hloupé odpovědi, ale podle toho, že odpoví pokaždé jinak.

Use case určuje techniku

Use case	Typicky
Získat informace	krátký prompt, silný model s vyhledáváním
Rozvést, porovnat	role, publikum, „oponuj“
Zpracovat (shrnout, vytáhnout)	přesný formát, klidně malý model
Vygenerovat, upravit	příklady a styl

Každá instrukce má **sloveso, rozsah a co nedělat**: „Shrň do 3 vět; každá věta jedno tvrzení; nic nepřidávej.“

Anatomie – osm částí

Část	K čemu	Malý model
Instrukce	sloveso, objekt, rozsah	začátek **i konec**
Role „on“	slovník, míra detailu	jedna věta s chováním
Role „já“	publikum	jako pravidlo
Kontext	co nelze uhodnout	škrtat první
Příklady	formát, tón, logika	**nejsilnější** (2–3)
Styl, jazyk	jak to zní	střídmě
Formát	jak to vypadá	vždy, přesně
Výstrahy	délka, „NEVÍM“	pozitivně

Pořadí: instrukce první → data uprostřed → formát a omezení na konci. Ne všechny části jsou potřeba pokaždé.

POZOR Malé modely negace přeslechnou: místo „nepiš úvod“ → „začni první odrážkou“; místo „nevymýšlej“ → „cituj větu z textu“.

Syntaxe – kdy jakou

Syntaxe	Kdy
Markdown	nadpisy a odrážky; nejběžnější; tabulky na výstup
XML značky	nejjasnější oddělení instrukcí a dat
JSON + schéma	výstup pro program; spolehlivé i u malých modelů
YAML	spíš konfigurace než výstup modelu
`###`, `---`	hranice sekcí; emoji jen pro lidi (stojí tokeny)

- **Data vždy ohraničit** (`<text>…</text>` nebo ```` ``` ````), jinak model zamění data za instrukce.
- **Jedna syntaxe na prompt** – míchání malý model zmate.
- Formát výstupu = formát, který umíte **parsovat**.

```
<instrukce>Shrň text v <text> do tří vět.
Každá věta jedno tvrzení. Nic nepřidávej.
</instrukce>
<text> ... </text>
<format>Tři věty, každá na řádku.</format>
Zopakuji: tři věty, jen z <text>.
```

Šablona promptu

```
## Úkol          {sloveso + objekt + rozsah}
## Role          Ty: {…}  Já: {…}
## Kontext       {jen to, co mění odpověď}
## Postup        1. … 2. … 3. zkontroluj {kritérium}
## Formát        {tabulka / JSON / N odrážek}
```

- Psát **mimo chat** (editor, Obsidian, VS Code); chat je pro spuštění.
- Ukládat jako soubory (`prompts/shrnuti.md`) s datem, modelem a poznámkou; **verzovat v gitu**.
- Proměnné `{text}`, `{jazyk}` → stejný prompt pro sto vstupů.
- **Promptová peněženka:** 10 promptů, které opravdu používáte.
- **Evoluce:** ad hoc → prázdné šablony → vyplněné šablony (stačí pro sebe) → **šablony s testy** (nutné, když prompt volá program nebo jde o malý model).

Malý model – co to je (stav k datu)

- **1–8B parametrů, kontext 4K–32K** (frontier 200K–1M); Q4, 2–6 GB, notebook, offline.
- Příklady: Gemma 4 E2B / E4B, Phi-4-mini, Ministral 3 3B / 8B, Qwen3 4B / 8B, gpt-oss-8b, Nemotron 3 Nano.
- **Slabiny:** složené instrukce (splní první), negace, výjimky, **lost in the middle**, méně faktů, horší čeština.
- **Exceluje:** klasifikace, extrakce do JSON, přepis formátu, shrnutí kratších textů – tisíckrát, levně, offline.

Rozpočet kontextu

Okno	≈ slov CZ	≈ A4	Vejde se
4K	1 800	4	prompt + 1 strana + krátká odpověď
8K	3 500	8	few-shot + 1 dokument + JSON
32K	14 000	30	víc dokumentů, střed se ztrácí
128K	55 000	120	u malých modelů jen na papíře

Instrukce + příklady ≤ 30 % · data ≤ 45 % · rezerva na odpověď ≥ 25 %

- Odpověď, která se nevejde, se **utne bez varování**.
- Systémový prompt se počítá pokaždé (50řádková persona v 8K = 10 % na nic).
- Čeština ≈ **1,5–2× víc tokenů** než angličtina. Tokeny počítat (tokenizer, tiktoken, Ollama `/api/tokenize`), ne odhadovat.

Techniky

Technika	Princip	Kdy	Malý model
Zero-shot	úkol bez příkladů	jednoduché, rychlé dotazy	běžné úlohy ano; záleží-li na formátu, přidat příklad
One-shot	jeden příklad výstupu	struktura, tón, méně běžná úloha	nejlevnější zlepšení; končit „Výstup:“
Few-shot	několik příkladů	jasný vzor, specifická úloha	**nejsilnější**; 3–5, vyvážené třídy, poslední řádek nedokončený; > 8 v 8K ubírá datům
Chain of thought	krok za krokem	logika, výpočty, více kroků	explicitní kroky ano; obecné „přemýšlej“ až od ~7B; thinking modely nechat
Tree of thoughts	víc větví, porovnat, vybrat	scénáře, strategie	rozložit do 3 promptů (varianty → hodnocení → výběr)
Meta prompting	AI píše / hodnotí prompt	opakovaně použitelné prompty	psát na velkém modelu, spouštět na malém
Sokratovské	řetěz otázek	hlubší analýza, argumentace	4 otázky v promptu ano, 8 ne; jinak po jedné
Emotional	„je to pro mě důležité“	experimenty	efekt neměřitelný; **historická kuriozita**
Generování znalostí	nejdřív fakta, pak odpověď	oddělit fakta od doporučení	dva prompty; fakta z kroku 1 ověřit
PAL	model napíše program, počítá interpret	výpočty, data, přesná odpověď	**nejlepší pro malé**: neumí násobit, napíše one-liner
Reflexe	kritika vlastního výstupu	kontrola proti zdroji	dvě relace, kritika ideálně na větším modelu

Deset pravidel pro malý model

1. **Jeden úkol na prompt** – řetězit programem, ne „a potom“.
2. **Instrukce na začátek a zopakovat na konec.**
3. **Pozitivně, ne negací.**
4. **Příklad místo popisu** (2–3 ve formátu výstupu).
5. **Krátká role** – chování, ne životopis.
6. **Formát přesně** + „odpověz jen…“.
7. **JSON se schématem** – vynucený formát místo prosby.
8. **Dokument rozdělit** – kus ≤ 45 % okna.
9. **Instrukce anglicky, data česky** – výstup lze žádat česky.
10. **Teplota 0 a testovat** – 10 vstupů před nasazením.

Extrakce a klasifikace lokálně

```
r = ollama.chat(model="gemma4:e4b",
  messages=[{"role": "user",
    "content": PROMPT.format(t=t)}],
  format=SCHEMA,              # JSON schema
  options={"temperature": 0})
json.loads(r.message.content)
```

- `format` = schéma → model nevrátí nevalidní JSON; `temperature 0` → stejný vstup, stejný výstup.
- Neznámé hodnoty → `null`; sledovat, kolikrát ho model použil.
- **Klasifikace few-shot:** 4 třídy, 4 vyvážené příklady, poslední `Label:` nedokončený, „label only“ + stop sequence `\n`.
- Test 40 tiketů s ručním štítkem; **pod 90 %** přidat příklady nebo změnit třídy.

Dlouhý dokument: map-reduce

1. **Map:** každý kus stejným promptem („3 bullets, facts only, keep numbers and names“).
2. **Reduce:** spojit shrnutí („5 bullets, no duplicates; if parts conflict, say so“).

- Dělit po odstavcích / kapitolách, ne po N znacích; překryv 1–2 věty.
- Výsledky ukládat průběžně; map na malém, reduce klidně na větším modelu.
- Otázky nad dokumentem = už **RAG** (téma 5).

Jak testovat prompt

1. **10 testovacích vstupů** – reálné, různorodé, 2 ošklivé (prázdný, mimo téma) + očekávaná odpověď.
2. **Spustit v cyklu** skriptem, teplota 0, výstupy uložit k datu a modelu.
3. **Spočítat jedno číslo** – shoda u klasifikace, validní JSON u extrakce, klíčová slova u shrnutí.
4. **Změnit jednu věc** a porovnat čísla, ne dojmy.
5. Počítat s **nedeterminismem** (stejný vstup 5×) – nástroje: promptfoo, pytest.

POZOR Halucinace a bias prompt **zmenší, nezruší** – hranice („NEVÍM“, citace) a lidská kontrola. Malý model, který hodnotí sám sebe, si dá jedničku.

Na zapamatování

TIP **Nástroj × model × prompt × technika = výsledek.** Když nesedí, měňte jednu proměnnou – nejčastěji prompt, pak model.

Zdroj: přednáška Prompty (7LLMA-2-prompty-v2.1.pptx), stav k 22. 9. 2026.